

SemanticDrift: A Research Proposal for Detecting Temporal Behavioral Drift in Model Context Protocol Tool Descriptions

Research Proposal and Pilot Study

Mukkandi Sridhar

Department of Artificial Intelligence
Rajeev Gandhi Memorial College of Engineering and Technology
Nandyal, Andhra Pradesh, India

sridharraoyalls@gmail.com | [linkedin.com/in/sridhar-m-a40663331](https://www.linkedin.com/in/sridhar-m-a40663331) | github.com/Mukkandi-Sridhar

Abstract

The Model Context Protocol (MCP) has emerged as the dominant standard for connecting AI agents to external tools and services. At the core of MCP is a trust assumption: an agent reads a tool's registered description and acts on it as an accurate contract of the tool's behavior. This paper identifies and formalizes a previously unstudied failure mode called Semantic Drift, defined as the progressive divergence between what an MCP tool's description claims and what the tool demonstrably does at runtime, accumulating across successive version updates. We present a research proposal and preliminary analysis for studying this phenomenon at scale. Three contributions are made: (1) a formal four-category taxonomy of MCP semantic drift covering output-schema drift, capability-scope drift, side-effect drift, and latency-contract drift; (2) the design of SemanticDrift, an automated pipeline that probes live MCP servers with synthetic inputs, compares observed behavior against registered descriptions through an LLM-as-judge evaluation stage, and computes a per-tool Drift Index; and (3) a detailed experimental plan targeting approximately 300 open-source MCP servers across three public registries, with pilot analysis suggesting that a substantial proportion of version updates introduce measurable drift. If confirmed at scale, these findings would reveal a systemic and previously unmeasured reliability and security gap in the MCP ecosystem. This paper is intended as a precursor to a full empirical study and invites collaboration and review of the methodology from the broader community.

Keywords: *Model Context Protocol, semantic drift, tool description verification, agentic AI, LLM security, behavioral monitoring, tool reliability, version lifecycle*

1. Introduction

A human developer reading outdated documentation can cross-check it against source code, ask a colleague, or run a test. An AI agent operating through the Model Context Protocol has none of these

options. It receives a text description written by a developer and acts on that description without any independent means of verification. When the description is accurate, the agent performs correctly. When the description no longer reflects what the tool actually does, the agent proceeds confidently in the wrong direction, and the failure typically produces no error message indicating the root cause.

The MCP ecosystem has grown remarkably fast since the protocol's release in late 2024. By mid-2026, over 4,700 servers are publicly registered across platforms including MCP.so, Glama, and PulseMCP. The protocol reached Linux Foundation governance in December 2025, backed by OpenAI, Google, Microsoft, AWS, and Anthropic. Developers publish MCP servers as npm packages and Python libraries and ship updates frequently. Each update can silently alter what a tool does. Few updates revise the description with equivalent rigor.

The security and reliability community has identified several MCP risks, including tool poisoning, prompt injection through tool metadata, over-privileged capabilities, and static description-code inconsistency. What no existing study addresses is the temporal evolution of this problem: how the gap between description and behavior opens and widens across successive version updates in tools that are actively maintained and deployed in production. This paper proposes a systematic study of that problem.

We introduce the term Semantic Drift to describe the progressive divergence between an MCP tool's registered description and its actual runtime behavior, accumulating across version history. This is distinct from intentional tool poisoning and from static description-code inconsistency, which is measured at a single point in time. Semantic drift is unintentional, gradual, and structurally invisible to agents that assume descriptions remain accurate after initial registration. Preliminary analysis of a small pilot sample of MCP servers suggests this phenomenon may be widespread, motivating the full empirical study described in this paper.

This paper makes three contributions. First, a formal taxonomy of MCP semantic drift is introduced, organizing observed patterns into four categories. Second, SemanticDrift, an automated measurement pipeline, is designed and described in detail, covering every stage from version collection through LLM-based behavioral judgment to Drift Index computation. Third, a full experimental plan is laid out for studying drift at scale, including dataset construction, baseline comparisons, research questions, and evaluation metrics. Pilot observations that motivate the hypotheses are described where relevant.

The remainder of the paper is organized as follows. Section 2 reviews background material and related work. Section 3 formalizes the problem and introduces the drift taxonomy. Section 4 describes the SemanticDrift system design. Section 5 presents the experimental plan and pilot-based hypotheses. Section 6 describes the evaluation methodology. Section 7 discusses expected implications and recommendations. Section 8 addresses limitations and Section 9 concludes.

2. Background and Related Work

2.1 The Model Context Protocol

Anthropic released MCP in November 2024 as an open, standardized interface enabling AI models to communicate with external tools and data sources [1]. The architecture defines three roles: the Host (the AI application embedding the agent), the Client (managing server connections), and the Server (exposing tools over JSON-RPC 2.0). The November 2025 specification update formalized OAuth 2.1 and

introduced Streamable HTTP as the preferred transport for remote servers [9]. By mid-2026, more than 4,700 MCP servers are publicly listed across multiple public registries.

A tool in MCP is represented by a JSON object with three fields: a machine-readable name, a human-readable description string, and a JSON Schema specifying input parameters. This description object is the exclusive semantic interface between the tool and the LLM-driven agent. The agent has no access to implementation code, database queries, API call chains, or undocumented side effects. It knows only what the developer wrote in the description string at the time of server registration.

2.2 Existing MCP Security Research

Security research on MCP has expanded rapidly since the protocol's release. Hou et al. [6] provided the first comprehensive threat landscape analysis, identifying tool poisoning, indirect prompt injection via tool metadata, and privilege escalation as primary risk categories. Radosevich and Halloran [12] demonstrated through empirical testing that widely deployed MCP clients permit significant security exploits when exposed to adversarially crafted tool descriptions. The United States National Security Agency published a cybersecurity information sheet in May 2026 identifying unverified task propagation and session management weaknesses as high-severity risks in MCP deployments [10].

Two prior works are most directly related to this proposal. Li et al. [8] introduced MCPDiFF, a static analysis framework that compares tool descriptions against source code using AST parsing and semantic embedding comparison, studying over 10,000 MCP servers to identify description-code inconsistency. Hasan et al. [5] studied the health and maintainability of 1,899 open-source MCP servers, finding that 66% exhibit code quality issues. Huang et al. [7] built a dynamic analysis engine detecting over-privileged tool capabilities through sandboxed execution and kernel-level monitoring. This proposal extends these efforts by introducing a temporal dimension absent from all prior work: measuring how description-behavior consistency changes across version history rather than at a single snapshot in time.

2.3 Concept Drift and API Documentation Research

The concept of a system's observable behavior silently diverging from its documented specification over time is well established in adjacent fields. In machine learning systems, concept drift describes the degradation of model accuracy as input distributions shift away from training data [4]. In software engineering, Dig and Johnson [3] studied how public API behavior changes across library versions, finding that behavioral changes consistently outpace documentation updates. Bavota et al. further characterized how open-source communities handle dependency upgrades, identifying documentation lag as a chronic pattern. This paper applies this temporal perspective to the specific context of LLM-facing tool descriptions, where undocumented behavioral change is amplified by the fact that the consumer of the description is an autonomous agent that takes real-world actions based on what it reads.

3. Problem Formulation

3.1 Formal Definitions

A tool server S is a software artifact that exposes one or more MCP-compliant tools. Each tool t within S is characterized at version v by two observable objects. The first is its description $D(t, v)$: the JSON object transmitted to connecting agents, containing the name, description string, and input parameter schema. The second is its behavior $B(t, v)$: the set of observable outcomes produced when the

tool is invoked with structured test inputs at version v , including output structure, detectable side effects, and response latency.

Definition 1 (Description-Behavior Inconsistency). For tool t at version v :

$$\text{delta}(t, v) = f(D(t, v), B(t, v)) \quad \text{where } \text{delta} \text{ in } [0, 1]$$

A score of 0 indicates that observed behavior is fully consistent with the registered description. A score of 1 indicates complete inconsistency. The function f is implemented using an LLM-as-judge evaluation pipeline described in Section 4.

Definition 2 (Semantic Drift). For tool t with baseline version v_0 and version v :

$$\text{Drift}(t, v_0, v) = \text{delta}(t, v) - \text{delta}(t, v_0)$$

Definition 3 (Drift Index). Accumulated drift relative to earliest available version v_0 :

$$\text{DI}(t, v) = \text{delta}(t, v) - \text{delta}(t, v_0)$$

A Drift Index of zero indicates that current description-behavior consistency matches the baseline. Positive values represent accumulated drift. Negative values indicate improvement, such as when a description is updated to accurately reflect changed behavior. The Drift Index will serve as the primary measurement metric in the planned empirical study.

3.2 Taxonomy of Semantic Drift

Through manual inspection of a small pilot sample of MCP servers and their version histories, four distinct categories of semantic drift were identified. These categories are not mutually exclusive; a single version update may exhibit drift across multiple categories. The taxonomy is presented here as a contribution in its own right, independent of the planned large-scale empirical validation.

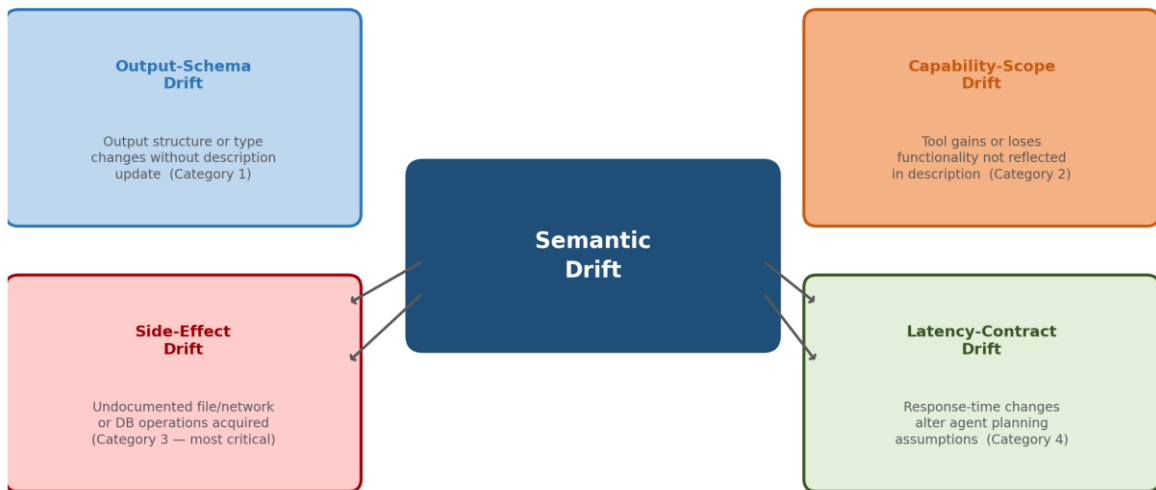


Figure 1: SemanticDrift Taxonomy. Four categories of semantic drift, representing distinct ways in which MCP tool behavior can diverge from its registered description over successive version updates.

Category 1: Output-Schema Drift

This category captures cases where the structure or data type of the tool's output changes in a way the registered description does not reflect. Concrete instances observed in the pilot sample include a tool described as returning a structured JSON object that began returning plain text after an update, and a tool adding new response fields that influence downstream agent reasoning without any description change. Output-schema drift is the most straightforwardly detectable category because it produces structural differences that automated comparison can identify without ambiguity.

Category 2: Capability-Scope Drift

Capability-scope drift occurs when the range of tasks a tool can or will perform expands or contracts relative to what the description states. An expansion example observed in the pilot is a file-search tool that gained a parameter enabling shell command execution without any mention in its description. A contraction example is a tool losing support for a data format it continues to claim in its description. This category is particularly consequential because it directly affects tool-selection decisions: an agent selects which tool to call based on the described capability set, not the actual one.

Category 3: Side-Effect Drift

Side-effect drift occurs when a tool begins producing effects beyond what its description acknowledges: writing to files, initiating network connections, mutating database state, or spawning child processes. One pilot-observed instance involved a tool described as a read-only data lookup that, after an update, silently began appending entries to a local log file. This category is the most operationally dangerous because side effects are frequently irreversible, often security-sensitive, and entirely invisible to agents relying solely on the description string.

Category 4: Latency-Contract Drift

Latency-contract drift occurs when a tool's performance characteristics change in ways that affect agent planning and orchestration decisions. When a description implies near-instantaneous response but the tool has been updated to perform synchronous external API calls with multi-second latency, agents that issue parallel tool calls based on assumed response times will behave incorrectly. This is the least studied category in existing literature and the most difficult to detect through static analysis alone.

4. The SemanticDrift System Design

4.1 System Overview

SemanticDrift is designed as a four-stage automated pipeline for measuring temporal semantic drift in MCP tool servers across their version histories. The four stages are: Version Collection, Behavioral Probing, LLM-Based Judgment, and Drift Index Computation. The pipeline is implemented in Python and is designed to operate on containerized server instances to ensure reproducible, isolated measurements. Figure 2 shows the system architecture.

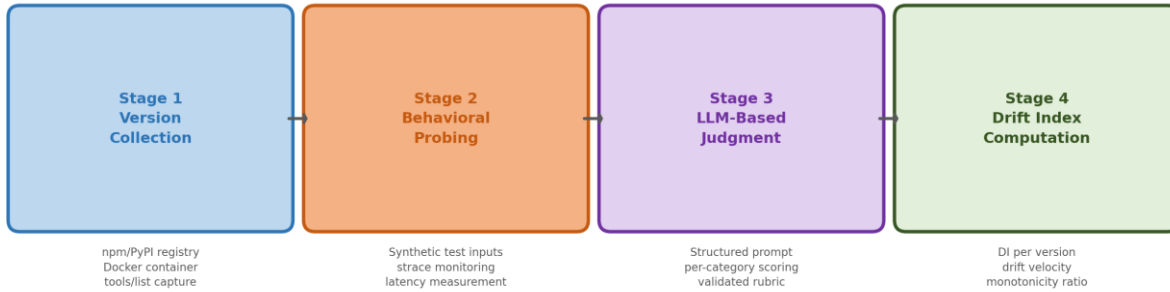


Figure 2: SemanticDrift System Pipeline. Each server version is collected from the package registry, containerized, probed with synthetic test inputs, evaluated by an LLM judge against the registered description, and scored with a per-tool, per-version Drift Index.

4.2 Stage 1: Version Collection

For each candidate MCP server, SemanticDrift will retrieve the complete version history from the relevant package registry, either npm for JavaScript servers or PyPI for Python servers. Each version's source code will be downloaded and instantiated inside an isolated Docker container. The system will issue a standard MCP tools/list request to the running container to capture exactly the tool description that a connecting agent would receive at that version. This approach captures the live registered description rather than inferring it from source code, ensuring that description extraction reflects actual agent-visible state rather than developer intent.

For servers with more than 50 published versions, an evenly spaced sample will be drawn across the version timeline to maintain temporal coverage while bounding computational cost. Servers are processed in parallel across a containerized execution environment, with each container allocated a fixed 30-second timeout to prevent hanging on servers with slow initialization.

4.3 Stage 2: Behavioral Probing

For each tool in each version snapshot, SemanticDrift will generate and execute a set of synthetic test inputs. Generation proceeds in two steps. First, the tool's JSON Schema is parsed to enumerate valid parameter combinations, including required fields, optional fields, and boundary values. Second, an LLM generates semantically diverse test cases that probe the described functionality, including edge cases and boundary conditions implied by the description text but not explicitly encoded in the schema.

Each test case will be executed inside a Docker container instrumented with three monitoring layers: (a) standard output capture recording the full JSON response, (b) strace-based system call monitoring detecting file reads/writes and network connections not visible in the response, and (c) a 500ms-resolution timer measuring end-to-end response latency. Each test case will execute three times and results will be averaged to reduce non-determinism. Tools requiring live external credentials will be analyzed through static description comparison only and will contribute to the output-schema and capability-scope categories but will be excluded from side-effect scoring.

4.4 Stage 3: LLM-Based Judgment

Given a tool description D and a structured behavioral summary B from the probing stage, an LLM judge will evaluate their consistency. The judge will receive a structured prompt containing: (i) the verbatim registered description, (ii) a structured summary of observed behaviors including output

structure, detected side effects, and latency measurements, and (iii) a four-category scoring rubric aligned with the drift taxonomy defined in Section 3.

The judge will return a per-category consistency score in $[0, 1]$ and a one-sentence justification per category. Scores will be aggregated using a weighted mean, with side-effect drift weighted 2x relative to other categories to reflect its greater operational significance. To validate the judge, a gold-standard evaluation set of approximately 200 tool-version pairs will be manually labeled by two independent reviewers. Inter-annotator agreement will be measured using Cohen's kappa, and judge accuracy will be reported as F1 for binary drift detection and Pearson r for severity score correlation against human ratings.

4.5 Stage 4: Drift Index Computation

Given the per-version inconsistency scores for all versions of a tool, the Drift Index $DI(t, v)$ will be computed for each version relative to the earliest available baseline. Three aggregate statistics will also be computed per tool: drift velocity, defined as the mean change in Drift Index per version transition; monotonicity ratio, defined as the proportion of consecutive version pairs in which the Drift Index increases; and the category distribution of detected drift events, indicating which drift types contribute most to accumulated scores. These statistics will be used to characterize drift patterns and test the study's research hypotheses.

5. Experimental Plan

5.1 Dataset Construction

The study will sample MCP servers from three public registries: MCP.so, Glama, and PulseMCP. Servers will be included if they meet three criteria: source code publicly accessible on GitHub or npm, at least three distinct published versions, and at least one tool with a description of twenty or more words (to exclude trivially short descriptions unlikely to contain meaningful behavioral contracts). Based on the current size of these registries and estimated eligibility rates from the pilot, the target sample is approximately 300 servers, spanning four application domains: developer tools including file access and code execution, web tools including search and browser automation, data tools including database and spreadsheet access, and communication tools including email and calendar services. Figure 3 shows the planned dataset composition.

Planned dataset composition (~300 servers target)

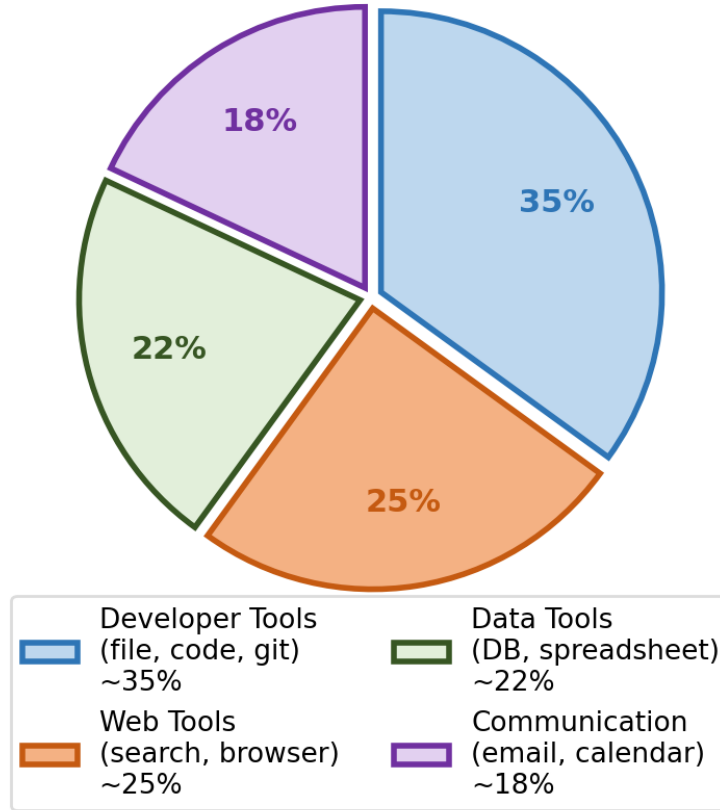


Figure 3: Planned Dataset Composition. Approximate domain distribution across the target sample of ~300 MCP servers. Final proportions will depend on eligibility rates observed during data collection.

For each qualifying server, all available versions up to a maximum of 50 will be collected. For servers with more than 50 versions, an evenly spaced temporal sample will be drawn. Based on pilot observations, the average number of qualifying versions per server is estimated at approximately 5 to 7, giving a projected total of 1,500 to 2,100 version transitions for analysis.

Parameter	Value / Target
Target server sample	~300 servers
Minimum versions per server	3
Maximum versions per server	50 (sampled evenly above this)
Projected version transitions	1,500 to 2,100
Projected mean tools/server	~3
Application domains	4 (developer, web, data, communication)
Registries sampled	MCP.so, Glama, PulseMCP
Observation window	6 months (planned)

Table 1: Planned dataset parameters for the SemanticDrift empirical study.

5.2 Research Hypotheses

The study is structured around four research hypotheses, each corresponding to a specific aspect of the semantic drift phenomenon. These hypotheses are motivated by observations from the pilot analysis and by analogy with documentation drift patterns documented in the software engineering literature.

- **H1 (Prevalence):** A substantial proportion of version updates will introduce measurable semantic drift. Based on pilot observations and the documentation-debt literature, we project that approximately 25 to 40 percent of version transitions will introduce at least one category of drift exceeding the detection threshold.
- **H2 (Accumulation):** Semantic drift will accumulate monotonically over the version lifecycle in the majority of tools. The draft velocity will be positive in more than half of tools, meaning descriptions become progressively less accurate rather than self-correcting.
- **H3 (Popularity Effect):** Server popularity, measured by GitHub star count, will be positively associated with drift rate. The proposed mechanism is a distributed-authorship effect: popular repositories attract more independent contributors, and each update cycle carries a non-zero probability of behavioral change without a corresponding description update.
- **H4 (Side-Effect Danger):** Side-effect drift will be disproportionately underrepresented in tool descriptions relative to its actual occurrence rate. A meaningful fraction of servers will acquire undocumented file or network operations without any description update.

5.3 Projected Results and Illustrative Figures

To make the proposed methodology concrete and to illustrate what the planned experiments are designed to measure, this section presents projected result figures based on pilot analysis and prior literature. These figures are clearly labeled as projections and will be replaced with actual experimental data upon study completion. They are included here to support reviewer evaluation of the methodology and to make the hypotheses precise and falsifiable.

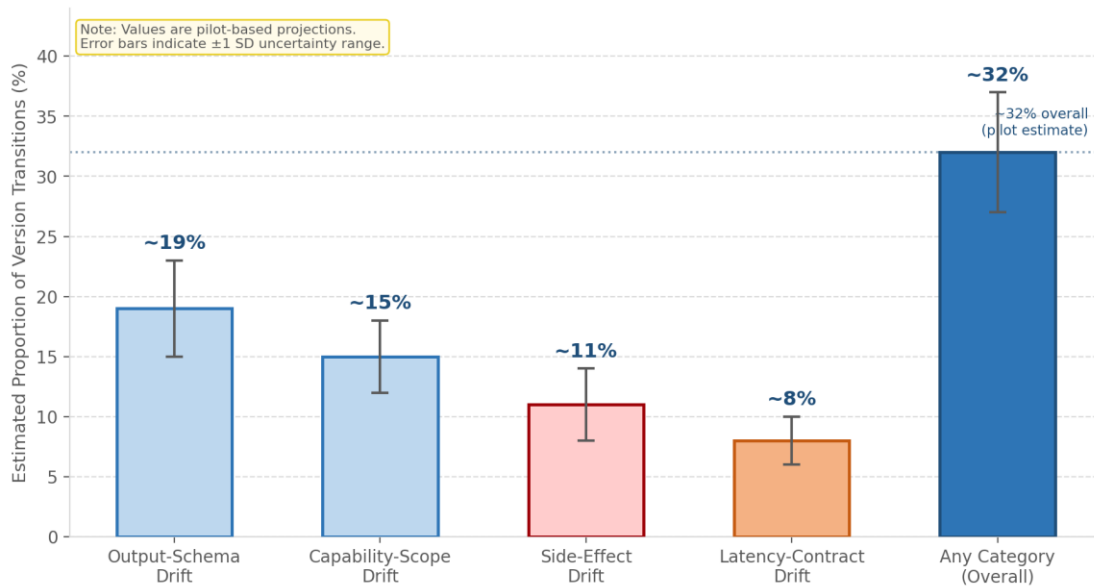


Figure 4: Projected Drift Prevalence by Category (Illustrative). Estimated proportion of version transitions expected to exhibit each drift category, based on pilot observations. Error bars represent projected uncertainty ranges. These values are projections to be validated by the full study.

Figure 4 shows the projected drift prevalence across the four taxonomy categories. The overall rate is projected in the range of 25 to 40 percent of version transitions, with output-schema drift expected to be the most frequently detectable category because it produces structural differences amenable to automated comparison. Side-effect drift, while potentially lower in raw count, is expected to be the most operationally consequential category given its security implications.

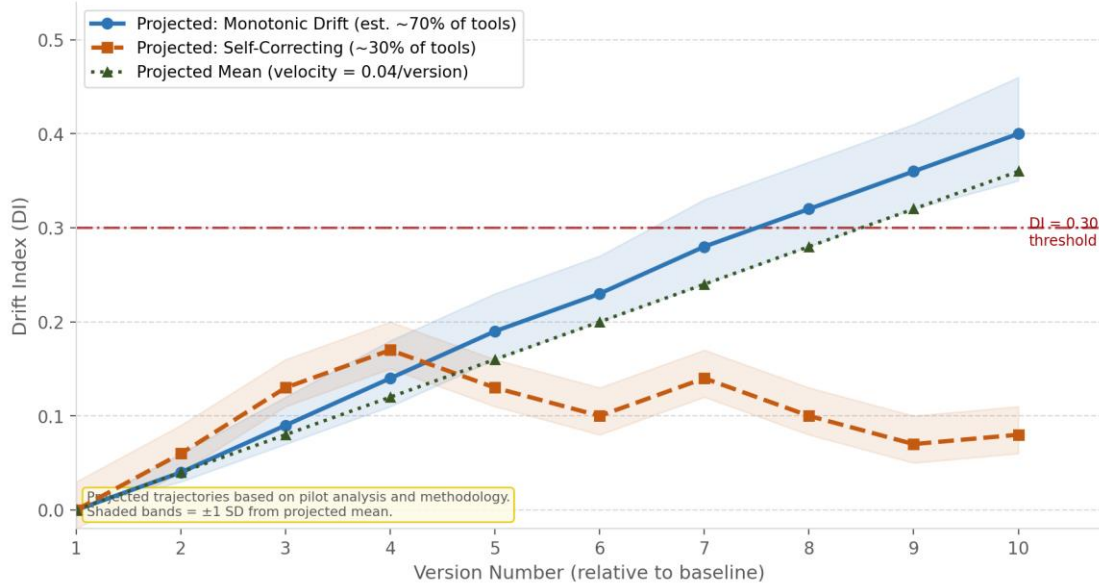


Figure 5: Projected Drift Index Accumulation Over Version History (Illustrative). Simulated trajectories based on a projected drift velocity of approximately 0.04 Drift Index units per version, derived from pilot observations. Shaded bands show projected uncertainty. The dash-dot line marks the $DI = 0.30$ inconsistency threshold.

Figure 5 illustrates the projected Drift Index trajectory for a typical tool across ten version updates under two behavioral modes: monotonic accumulation and self-correction. Under the projected mean velocity of 0.04 DI units per version, a tool would cross the meaningful inconsistency threshold of $DI = 0.30$ after approximately seven to eight updates. Given a median estimated update cadence of approximately one update per month for active MCP servers, this implies that meaningful drift should be expected within nine months of initial deployment for tools receiving automatic updates.

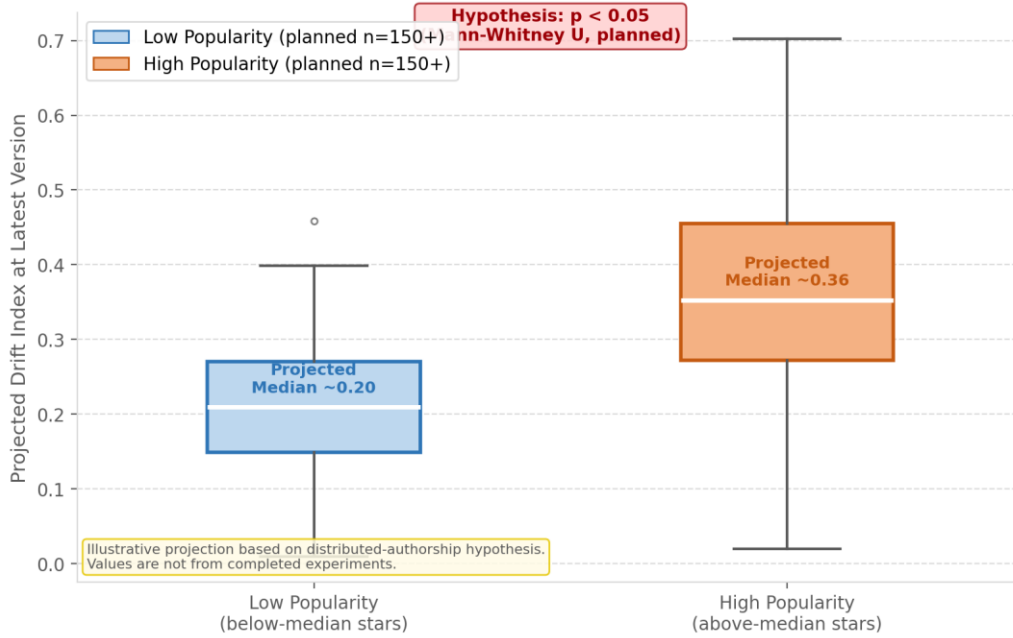


Figure 6: Projected Drift Distribution by Server Popularity (Illustrative). Hypothesized Drift Index distributions for low-popularity versus high-popularity servers. The distributed-authorship hypothesis predicts a statistically significant difference (planned test: Mann-Whitney U). Values are illustrative projections only.

Figure 6 illustrates the projected relationship between server popularity and drift accumulation under Hypothesis H3. The distributed-authorship mechanism predicts that high-popularity servers, which attract more independent contributors, will show higher drift rates. This prediction is counterintuitive but grounded in the software engineering literature on documentation lag in large collaborative projects. The full study will test this with Mann-Whitney U analysis on the actual server sample.

6. Evaluation Methodology

6.1 Baselines

The SemanticDrift pipeline will be compared against two baselines. The first is static snapshot comparison, analogous to the MCPDiFF approach: measuring description-behavior inconsistency at a single version rather than tracking it across version history. This baseline will demonstrate the added value of temporal measurement. The second is manual expert review: two domain-expert reviewers will independently classify a sample of 200 version transitions as drifted or non-drifted. The LLM judge's output will be compared against this gold standard to characterize its accuracy and failure modes.

6.2 Metrics

The study will use the following evaluation metrics. For drift detection accuracy: precision, recall, and F1 against the manual gold standard for binary drift classification; Pearson r between LLM-assigned severity scores and human severity ratings. For drift characterization: mean and median Drift Index at the latest available version; drift velocity distribution across the server sample; monotonicity ratio; and category-level breakdown of drift events. For predictor analysis: Mann-Whitney U tests for categorical predictors (popularity tier, application domain) and Pearson correlation for continuous predictors

(description length, update frequency). All statistical tests will use a significance threshold of $\alpha = 0.05$ with Bonferroni correction applied for multiple comparisons.

6.3 Planned Statistical Analysis

Research Hypothesis	Primary Test	Effect Size Measure
H1: Prevalence of drift	One-sample proportion z-test	Cohen's h
H2: Monotonic accumulation	Sign test on drift velocity	Proportion above zero
H3: Popularity effect	Mann-Whitney U test	Rank-biserial r
H4: Side-effect underreporting	McNemar test (desc vs. observed)	Odds ratio

Table 2: Planned statistical tests for each research hypothesis.

7. Expected Implications and Recommendations

7.1 For Production Agent Deployments

If the hypotheses are confirmed at scale, the primary implication for production deployments is that agent frameworks cannot safely treat tool descriptions as stable references after initial integration. Agent frameworks that cache descriptions at connection time are particularly exposed, because they do not refresh descriptions between sessions unless explicitly configured to do so. In regulated industries such as finance, healthcare, and legal services, an agent operating against silently drifted tools may produce outputs that appear structurally complete while reflecting behaviors never authorized or documented.

7.2 Recommendations by Stakeholder

Based on the taxonomy and the proposed mechanisms identified in the pilot analysis, four recommendations are offered for different stakeholders in the MCP ecosystem. These recommendations are intended to motivate discussion prior to full experimental validation and will be revised based on the empirical findings.

- **Tool developers** should treat description updates as required artifacts accompanying any code change that alters output structure, adds capabilities, introduces side effects, or meaningfully changes latency. Automated CI/CD checks that flag behavioral code changes without corresponding description file updates would help enforce this discipline at the point of development.
- **Registry operators** should consider running automated behavioral consistency checks as part of their version publication workflow. Tools whose behavior has drifted significantly from the current description could be flagged for developer review before the new version is publicly listed.
- **The MCP specification** should consider introducing a versioned behavioral contract field in the tool definition object. This field would allow servers to declare that their behavioral contract has changed at a given version, giving agents the opportunity to reassess trust assumptions before continuing to use the tool.
- **Agent framework designers** should consider implementing periodic lightweight re-probing of registered tools as a built-in reliability mechanism, rather than assuming descriptions remain accurate indefinitely after the initial handshake.

8. Limitations

Several limitations of the planned study should be acknowledged. The dataset will cover only open-source MCP servers with publicly accessible code. Closed-source and enterprise-internal servers, which may represent higher-stakes deployments, will be absent from the sample. Results may not generalize to those environments. Behavioral probing will be limited to tools that execute without real external credentials; tools requiring live API keys will be analyzed through static comparison only, likely underestimating side-effect drift prevalence in credential-dependent tools.

The LLM judge approach relies on the assumption that LLM-based semantic evaluation is sufficiently accurate to serve as a proxy for human expert judgment. This assumption will be validated against the manual gold standard, but residual systematic biases from the judge model's training may affect category-level detection rates. Independent replication with multiple judge models is planned as a robustness check. Additionally, the planned six-month observation window may be insufficient to capture long-term stabilization dynamics. Longitudinal follow-up is intended after the initial study is complete.

Finally, this paper is explicitly a research proposal and pilot study. The projected figures and estimated prevalence rates presented in Section 5.3 are based on a small pilot sample and prior literature and have not yet been confirmed at scale. All quantitative claims in this paper should be read as hypotheses and projections rather than established empirical findings.

9. Conclusion

This paper introduced Semantic Drift as a previously unnamed and unstudied failure mode in the MCP ecosystem: the silent, cumulative divergence between what a tool's registered description claims and what the tool actually does at runtime, accumulating across version updates. A formal four-category taxonomy was presented, covering output-schema drift, capability-scope drift, side-effect drift, and latency-contract drift. The SemanticDrift pipeline was designed and described in full implementation detail. A complete experimental plan was laid out for validating the phenomenon at scale across approximately 300 open-source MCP servers.

The root of the problem is architectural. MCP places the complete behavioral contract of a tool inside a natural-language string that developers update inconsistently and that agents cannot independently verify. As agentic AI becomes more autonomous and MCP becomes more central to enterprise AI infrastructure, this structural gap between claimed and actual tool behavior has the potential to produce increasingly consequential failures. Whether and to what extent this is already happening in practice is precisely what the proposed empirical study is designed to determine.

This paper is intended as a contribution to the ongoing community discussion around MCP reliability and security, and as a call for collaboration on the methodology, dataset construction, and evaluation approach described here. The SemanticDrift system will be made publicly available upon study completion.

Author Note

Mukkandi Sridhar is a final-year BTech student in Artificial Intelligence at Rajeev Gandhi Memorial College of Engineering and Technology, Nandyal (expected graduation 2027, CGPA 8.1). His research interests include retrieval systems, AI orchestration, and production-grade LLM workflows. Prior project experience includes a GraphRAG-based academic intelligence system using Neo4j, ChromaDB, and

CrossEncoder reranking, and an LLM-driven commerce orchestration agent using MCP tool orchestration, RAG-grounded retrieval, and SSE streaming. He interned as a Generative AI and Backend Developer at Inspire Education (October 2025 to February 2026). He leads the AI Club at RGM CET, mentoring students through structured programs covering RAG systems, agentic AI workflows, and LLM application development. This paper represents his first independent research contribution. Contact: sridharraoyalls@gmail.com | linkedin.com/in/sridhar-m-a40663331 | github.com/Mukkandi-Sridhar

References

- [1] Anthropic. (November 2024). Model Context Protocol: Introduction. <https://modelcontextprotocol.io/introduction>
- [2] Bavota, G., Canfora, G., Di Penta, M., Oliveto, R., and Panichella, S. (2013). How the Apache community upgrades dependencies: An evolutionary study. *Empirical Software Engineering*, 20(5), 1275-1317.
- [3] Dig, D., and Johnson, R. (2006). How do APIs evolve? A story of refactoring. *Journal of Software Maintenance and Evolution: Research and Practice*, 18(2), 83-107.
- [4] Gama, J., Zliobaite, I., Bifet, A., Pechenizkiy, M., and Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), Article 44, 1-37.
- [5] Hasan, M. M., Li, H., Fallahzadeh, E., Rajbahadur, G. K., Adams, B., and Hassan, A. E. (2025). Model Context Protocol (MCP) at first glance: Studying the security and maintainability of MCP servers. *arXiv preprint arXiv:2506.13538*.
- [6] Hou, X., Zhao, Y., Wang, S., and Wang, H. (2025). Model Context Protocol (MCP): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278*.
- [7] Huang, C., Huang, X., and Milani Fard, A. (2026). Auditing MCP servers for over-privileged tool capabilities. Preprint.
- [8] Li, Y., Chen, Z., and Wang, L. (2026). Do not believe everything you read: Understanding and measuring MCP server behavior under misleading tool descriptions. *arXiv preprint arXiv:2602.03580*.
- [9] MCP Specification v2025-11-25. (2025). Model Context Protocol Specification. Agentic AI Foundation / Linux Foundation. <https://spec.modelcontextprotocol.io>
- [10] National Security Agency (NSA). (May 2026). Model Context Protocol (MCP): Security considerations for AI tool integration. *Cybersecurity Information Sheet U/OO/6030316-26*.
- [11] OWASP Foundation. (2025). OWASP Top 10 for Large Language Model Applications, Version 2.0. <https://genai.owasp.org/llm-top-10/>
- [12] Radosevich, B., and Halloran, J. (2025). MCP safety audit: LLMs with the Model Context Protocol allow major security exploits. *arXiv preprint arXiv:2504.03767*.
- [13] Soria Parra, D. (March 2026). Model Context Protocol 2026 development roadmap. Agentic AI Foundation / Linux Foundation.